Design and Implementation of a Web System for Searching and Managing Courses - Course Finder

First Report

Brayan Alejandro Riveros Rodríguez
Cristian Santiago Ríos Vásquez
Carlos Andres Pescador Castro

Systems Engineering

Universidad Distrital Francisco José de Caldas

December 11, 2025

Abstract

Students often must search through numerous of courses, multiple instructors, and changing schedules across semesters, but information is often not centralized or in a palatable format, or in partially synchronized systems, making discovery and planning difficult. This project proposes a web-based system that consolidates course data and formalizes instructor-course relationships, separating authentication/authorization services from central domain-based course services, with non-functional objectives to integrate and centralize data, efficient search, and course-instructor associations. Workshops 1 and 2 established the central problem using business model canvas, user stories, CRC cards, and overall system design. Guided by Scrum as a development management methodology, preparing for sprint planning and traceability, this first report summarizes preliminary results and describes the project road map.

Contents

List of Figures

List of Tables

1 Introduction

1.1 Context and Motivation

Students often have no way to organize their semester schedules. At many institutions, including our own, course information is often poorly organized. This fragmentation makes it difficult for students to discover relevant offerings of interest, as well as the professors who teach them and their schedules. The result is avoidable problems during planning and registration periods, duplicate registrations, and delays in decision-making.

To address these issues, we propose a web system for searching and managing academic courses that consolidates essential data (course codes, titles, descriptions) and formalizes the relationships between professors and courses. The system targets three primary stakeholder groups: (i) *students*, who need fast, accurate search and discovery; (ii) *professors*, who is the associated with the courses; and (iii) *administrators users*, which is responsible for loading the data and managing the application. Our current work established the problem framing and initial design assets for the development phase.

From an engineering perspective, the project is intentionally designed to practice delivering end-to-end software within a semiannual schedule by implementing an agile approach.

1.2 Objectives

- Design and specify a reliable web-based system that allows key stakeholders to search, manage and maintain accurate information about courses and instructors.
- Define a clear and feasible road map for the implementation of the project that aligns with the identified problem context, stakeholder needs, and planning.
- Synthesize the context, stakeholders, and scope of the problem into a consistent baseline that unifies terminology and guides subsequent design and development activities.

1.3 Scope

This first report defines the requirements and design boundaries of the project at its current stage. It focuses on consolidating the conceptual and architectural foundations, including problem definition, stakeholder identification, user stories, CRC cards, high-level architecture, deployment views, and early user-interface sketches. The report also introduces preliminary elements for the data model and API contracts that will guide the implementation in the next phase. This scope explicitly includes documenting the

problem context, justifying the proposed solution, and establishing an initial road map under agile methodology using Taiga as the project management tool.

2 Related Work and Background

Students need digital systems to search and manage academic information such as course offerings, instructor assignments, and scheduling. However, on many occasions they find that in fragmented data or centralized in legacy applications that lack interoperability and transparency for stakeholders. In this context, academic management systems and course catalog platforms such as Moodle, Blackboard, and Google Classroom provide partial solutions, focusing primarily on learning management rather than the structural organization of academic offerings.

From a technical point of view, modern web-based architectures rely on Restful API s, micro-service decomposition, and standard authentication mechanisms such as JWT (JSON Web Token) to ensure secure and maintainable information exchange. These principles underpin the design of this project: a lightweight, modular system divided into a authentication service and a business This dual-back-end structure supports scalability, separation of concerns, and parallel development by different team members. Furthermore, adopting a continuous integration and deployment pipeline through tools like Docker and GitHub Actions allows the project to align with current DevOps practices that promote automation, repeatability, and early defect detection.

In terms of methodology, the use of Scrum supported support on the Taiga tool reflects an agile approach, iterative development of software engineering. Agile methods are widely used in both industry and academia to encourage adaptability, transparency, and stakeholder feedback. Previous studies and projects in software engineering have shown that integrating agile practices allows for better quality deliveries.

In summary, this work draws on principles from web-based academic management systems, requirement analysis and software design to propose a simple, fast, and verifiable solution tailored to the needs of the stakeholders.

3 Methodology

3.1 Process and Team Practices

The project will follows an agile software development approach based on the Scrum framework, adapted to the context of problem. The team uses **Taiga** as the main project

management tool to maintain the product backlog, define sprint goals, and track progress through user stories and tasks.

Roles are distributed collaboratively among the three members of the team. Carlos Andres Pescador Castro serve as the Product Owner and is the responsible for prioritizing the backlog and maintaining alignment with the defined requirements. Cristian Santiago Ríos Vásquez act as the Scrum Master, facilitating coordination among team members, ensuring adherence to Scrum events, and managing sprint reviews and retrospectives. Brayan Alejandro Riveros Rodríguez as the Development Lead, focusing on technical implementation, version control integration, ci/cd pipeline and testing coordination. All members contribute equally to documentation, gathering of requirements and continuous improvement activities.

Scrum ceremonies are conducted at a scaled frequency appropriate brief stand-up meetings are held twice per week (asynchronous when needed), sprint planning and retrospectives occur at the beginning and end of each workshop, and sprint reviews coincide with workshop submissions.

Version control and collaboration are managed through a shared Git repository, following a feature-branch workflow.

3.2 Scrum Implementation and Project Management

a

3.3 System Analysis and Design

Following the completion of the Workshops, the system analysis and design phase guide the transition from requirements specification to technical implementation. The main goal will be to ensure that the proposed architecture, domain entities, and component responsibilities are consistent with the user stories, acceptance criteria, and stakeholder needs identified during the analysis stage.

The analysis focus on consolidating the functional requirements derived from the user stories created in Workshop 1. Each user story will be mapped to a corresponding set of system components, defining clear inputs, processes, and outputs. CRC (Class–Responsibility–Collaborator) cards previously designed will serve as the foundation for assigning responsibilities among classes, while maintaining object-oriented design principles. This mapping will guarantee that each functional requirement has a direct

representation within the software model.

The system will adopt a modular architecture divided into two main backend services: an authentication and authorization service, and a core domain service for managing courses, professors, and their relationships. Both services will communicate through RESTful APIs, enabling independent deployment and scalability. Each backend will interact with its respective database—MySQL for authentication and PostgreSQL or MongoDB for the core domain—ensuring persistence and referential integrity across entities. The frontend web interface, developed with modern JavaScript framework, will consume these APIs to provide a cohesive user experience.

To formalize the design, UML diagrams are used to illustrate the system structure and interactions. A class diagram will describe the main domain entities (Professor, Course, and Link), their attributes, and associations. A deployment diagram will detail how components will be distributed across environments, showing the web client, API services, and database instances.

Non-functional requirements will also guide the design. The system will ensure security through JWT-based authentication and role-based access control, maintainability through modular coding practices, and performance through optimized database queries and API endpoints. Furthermore, logging and basic monitoring mechanisms will be defined to support future testing and evaluation.

This analysis and design process will establish a clear technical blueprint for the implementation stage to be developed in Workshop 3. By grounding each architectural decision in the previously validated requirements, the team will ensure traceability from stakeholder needs to system components and a solid foundation for the subsequent development and testing phases.

4 System Design and Architecture

4.1 User Stories and Requirements

The functional requirements of the system were captured through user stories following agile methodology principles. Each story represents a specific user need and includes well-defined acceptance criteria that guide both development and testing activities. The stories were structured using the standard template: "As a [user role], I want to [perform action], so that [achieve benefit]." This approach ensures that every feature delivers clear value to stakeholders and remains traceable throughout the development lifecycle.

Five core user stories form the foundation of the system functionality, divided into administrative capabilities, end-user search features, and role-based navigation. These stories were prioritized based on their dependency relationships and their impact on delivering value to all stakeholder groups. Table 1 provides a summary of all user stories with their dependencies and roles.

Table 1: Core User Stories for System Functionality

Story ID	**Story Name**	**User Role**	**Dependencies**
US_CF_01	Registration of New Professors	System Administrator	None
US_CF_02	Registration of New Courses	System Administrator	None
US_CF_03	Link Professor to Courses	System Administrator	US_CF_01, US_CF_02
US_CF_04	Search Courses by Professor and Vice Versa	System User	US_CF_03
US_CF_05	Home Page and Access Flow	Administrator /General User	US_CF_04, Auth Module

4.1.1 Administrative User Stories

Three stories address the System Administrator role, enabling the creation and management of the academic catalog that supports the search functionality.

US_CourseFinder_01: Registration of New Professors This story addresses the need to register professors in the system by capturing their personal and academic details through a dedicated registration form.

User Need: The administrator requires the ability to add new professors to the platform by entering their personal and academic information, ensuring the professor database remains current and complete.

Justification: Maintaining an up-to-date professor database ensures that users have access to accurate and complete information when consulting the system, supporting informed decision-making during course selection.

Key Validations: The system must validate all required fields before persisting data, including name, identification number, email, and specialty. Email uniqueness must be enforced to prevent duplicate registrations. The identification number must also be unique across all professor records.

Acceptance Criteria: The administrator can access the registration form from the main admin panel. All required fields are validated before saving data. The system displays a confirmation message upon successful registration. The new professor appears in the general professor list without requiring a page reload. Duplicate registrations based on email or identification number are prevented with appropriate error messages.

INVEST Characteristics: This story is independent and does not depend on other stories for development. It is negotiable in terms of specific fields and validation rules, which can be adjusted according to academic requirements. The story provides immediate value by establishing the foundational data required for the platform. Development effort is estimable based on form complexity and validation logic. The scope is small enough to complete within a single sprint. The story is testable through field validation tests, database persistence verification, and UI confirmation message checks.

US_CourseFinder_02: Registration of New Courses This story enables administrators to expand the academic catalog by adding new courses with comprehensive metadata.

User Need: The administrator wants to add new courses to the system, including details such as course name, code, description, credits, and category.

Justification: Expanding and maintaining the academic offerings ensures users have access to current course information, supporting effective academic planning and course discovery.

Key Validations: The system must validate course code uniqueness to prevent duplicate entries. All required fields must be completed before submission. If a professor is assigned during course creation, the relationship must be validated to ensure the professor exists in the database.

Acceptance Criteria: The administrator can access the course registration form from the main panel. All required fields are validated before saving. A confirmation message appears upon successful registration. The new course appears in the course list without

page reload. The system prevents creation of duplicate courses based on course code. If a professor is assigned, the relationship is correctly recorded in the database.

INVEST Characteristics: This story is independent and can be developed without dependencies on other stories. Fields and professor relationships are negotiable based on academic system needs. The story expands the information base, increasing platform utility for end users. Effort is estimable considering form implementation, validations, and database operations. The scope fits within a single sprint. Testing validates field requirements, database persistence, and relationship integrity.

US_CourseFinder_03: Link Professor to One or Multiple Courses This story implements the core relationship management functionality that establishes the many-to-many associations between professors and courses.

User Need: The administrator wants to link existing professors to one or more existing courses, establishing the teaching assignments that users will query.

Justification: Correct professor-course associations enable users to view accurate relationships when searching, supporting informed course selection based on instructor preferences.

Key Validations: Both the professor and course must exist in the database before linking. The system should prevent duplicate assignments of the same professor-course-group combination. If a course is already assigned to another professor in the same group, the system should warn the administrator or prevent the duplication based on business rules.

Acceptance Criteria: The administrator can access a dedicated interface for linking professors to courses. The system displays only existing professors and courses. The administrator can select one professor and one or multiple courses to link. Upon confirmation, the links are stored and visible in both entity profiles. Error messages appear if attempting to link non-existent entities. The system prevents or warns about duplicate course assignments. The administrator can unlink or modify existing relationships. A confirmation message appears after successful linking.

INVEST Characteristics: This story depends on US_CF_01 and US_CF_02, as both entities must exist before linking. The user interface and workflow are negotiable based on administrator feedback. The story provides core administrative control over relationships, ensuring data accuracy. Development time is estimable based on form

complexity, relationship logic, and validations. The scope fits within a single sprint for a small team. Testing validates UI interactions and backend relationship integrity.

4.1.2 End-User Functionality

Two stories address the primary use cases for general users: discovering course and professor information through search and accessing the system through role-appropriate entry points.

US_CourseFinder_04: Search Courses by Professor and Professors by Course
This story provides the core bidirectional search functionality that motivated the project development.

User Need: System users want to search for courses taught by a specific professor and search for courses to view their assigned professors.

Justification: Easy access to accurate professor-course relationships improves navigation and course discovery, helping users make informed academic decisions efficiently.

Key Validations: Search queries must handle partial text matches and ignore case sensitivity. The system must validate that search results accurately reflect the professor-course relationships established by administrators. Results must be complete and consistent with the database state.

Acceptance Criteria: Users can search by professor name and view all associated courses. Users can search by course name and view all assigned professors. The system displays clear, accurate, and complete results for each query. Search supports partial text matching and is case-insensitive. Friendly messages appear when no results are found. Search performance remains under two seconds for average query loads. The interface is responsive and adapts to desktop and mobile devices.

INVEST Characteristics: This story depends on US_CF_03, as accurate search requires established professor-course associations. Search filters, display format, and pagination are negotiable based on usability feedback. The story provides core functionality for navigating relationships, improving user experience significantly. Effort is estimable based on filter implementation, database queries, and frontend integration. The scope fits within a single sprint for a small development team. Testing validates that results accurately match linked data between professors and courses.

US_CourseFinder_05: Home Page and Access Flow for Administrator and User This story defines the main entry point of the system, providing role-appropriate navigation paths for administrators and general users.

User Need: Administrators want to access a dashboard summarizing key metrics with shortcuts to management actions. General users want immediate access to the search feature without unnecessary navigation.

Justification: Role-based entry points enable both user types to efficiently access the system according to their needs—administrators can manage data quickly, and users can immediately start searching.

Key Validations: The system must correctly identify the user's role after login. Role-based routing must function reliably, directing administrators to the dashboard and general users to the search interface. The homepage must load within acceptable performance thresholds (under two seconds). Administrative elements must be completely hidden from general user views.

Acceptance Criteria: The system redirects users based on their role: administrators to the dashboard, general users to the search page. The homepage loads correctly and displays personalized content. The admin dashboard shows relevant metrics (total professors, total courses, recent updates) and quick management options. The general user view focuses exclusively on search functionality. The interface is responsive and loads within performance thresholds. Navigation between homepage and other sections works without errors.

INVEST Characteristics: This story depends on US_CF_04 and the authentication module, as it requires functional search capabilities and role identification. Homepage layout, design elements, and dashboard widgets are negotiable based on feedback and usability tests. The story provides personalized entry points, ensuring efficient access and navigation for both roles. The scope is measurable by estimating layout design, routing logic, and conditional rendering. Implementation fits within a single sprint. Testing verifies redirection logic, role-based rendering, and proper section access.

4.1.3 Story Management and Traceability

The user stories were managed in the Taiga project management platform, where they were assigned story points, prioritized in the product backlog, and allocated to specific

sprints during sprint planning sessions. Each story's progress was tracked through states (New, In Progress, Done) and linked to specific tasks. Story points were estimated using planning poker sessions, with the team achieving consensus on complexity based on technical effort, uncertainty and integration requirements.

The dependency chain from administrative stories through search functionality to role-based navigation guided sprint planning, ensuring that foundational data management capabilities were delivered before user-facing features. This traceability ensures that every implemented feature can be validated against its original acceptance criteria and that test coverage aligns with user expectations. The acceptance criteria defined in each story directly influenced the test scenarios documented in Section 5 and the evaluation metrics presented in Section 6.

4.2 Web User Interface

This section presents the main artifacts developed during all the Workshops, which represent the foundation for the system implementation. Each artifact serves a specific purpose in documenting the design decisions and validating the feasibility of the proposed solution. The progression from conceptual models to concrete specifications ensures traceability from requirements to implementation.

Figure 1: Web User Interface (v1.0)

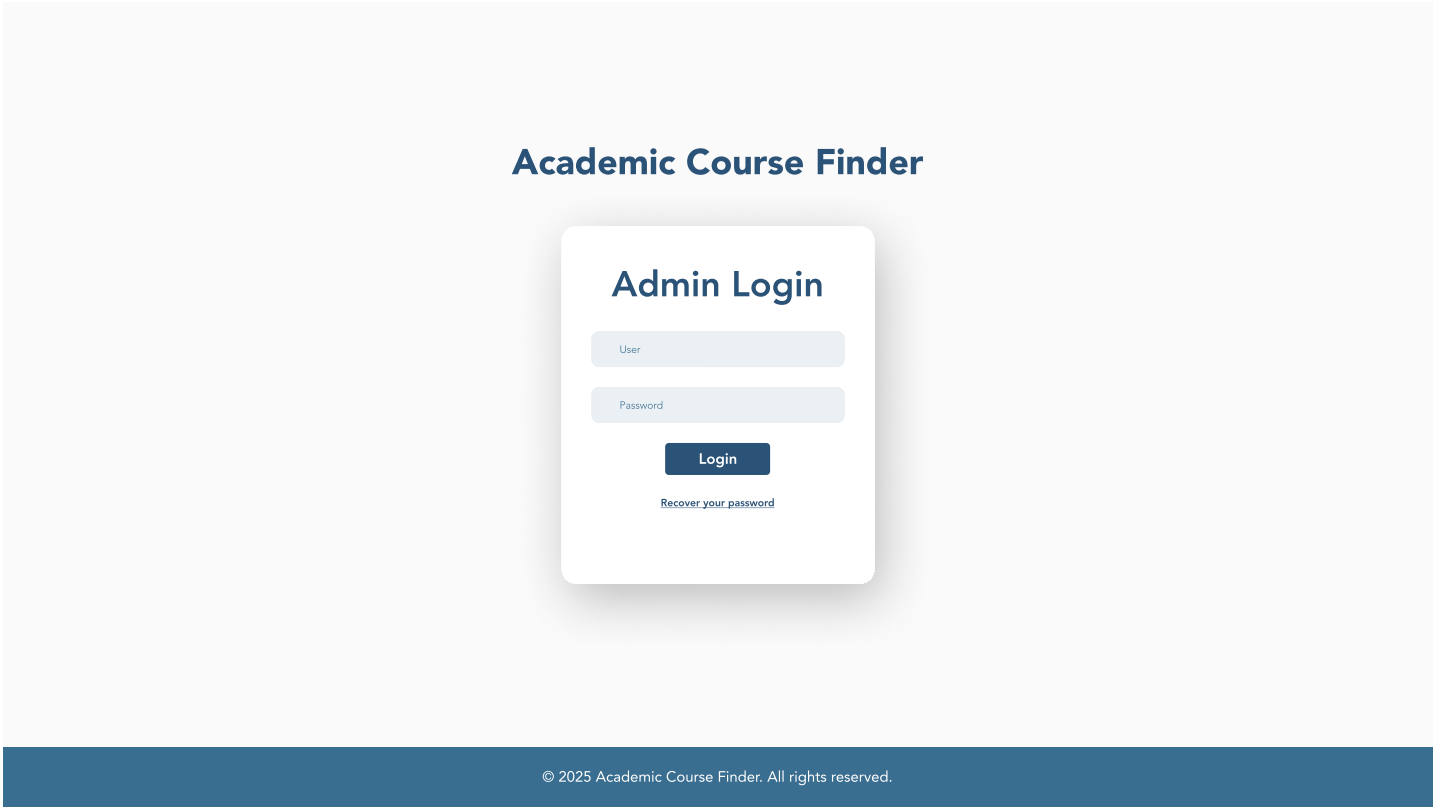


Figure 2: Web User Interface (v1.0)

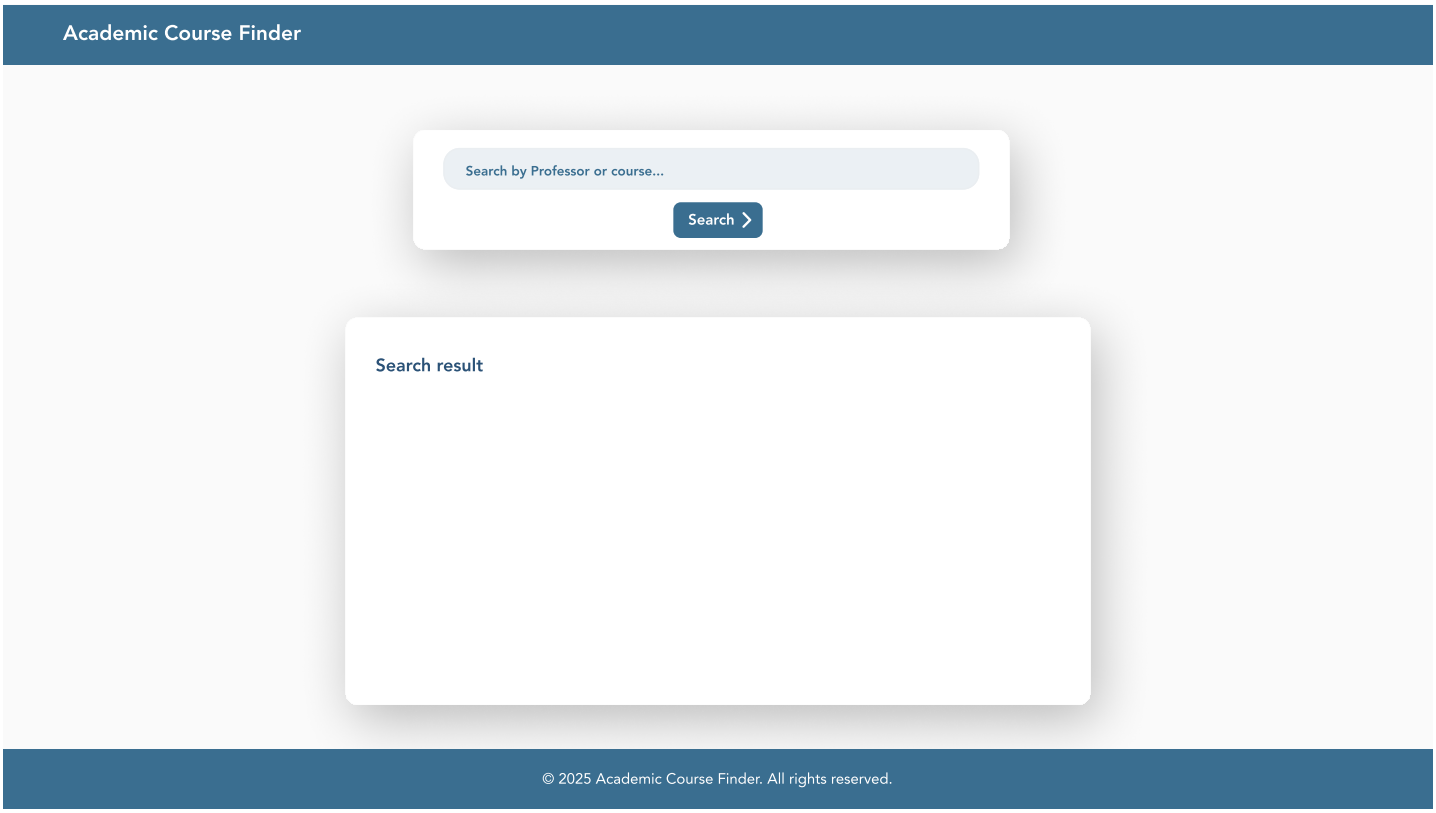


Figure 3: Web User Interface (v1.0)

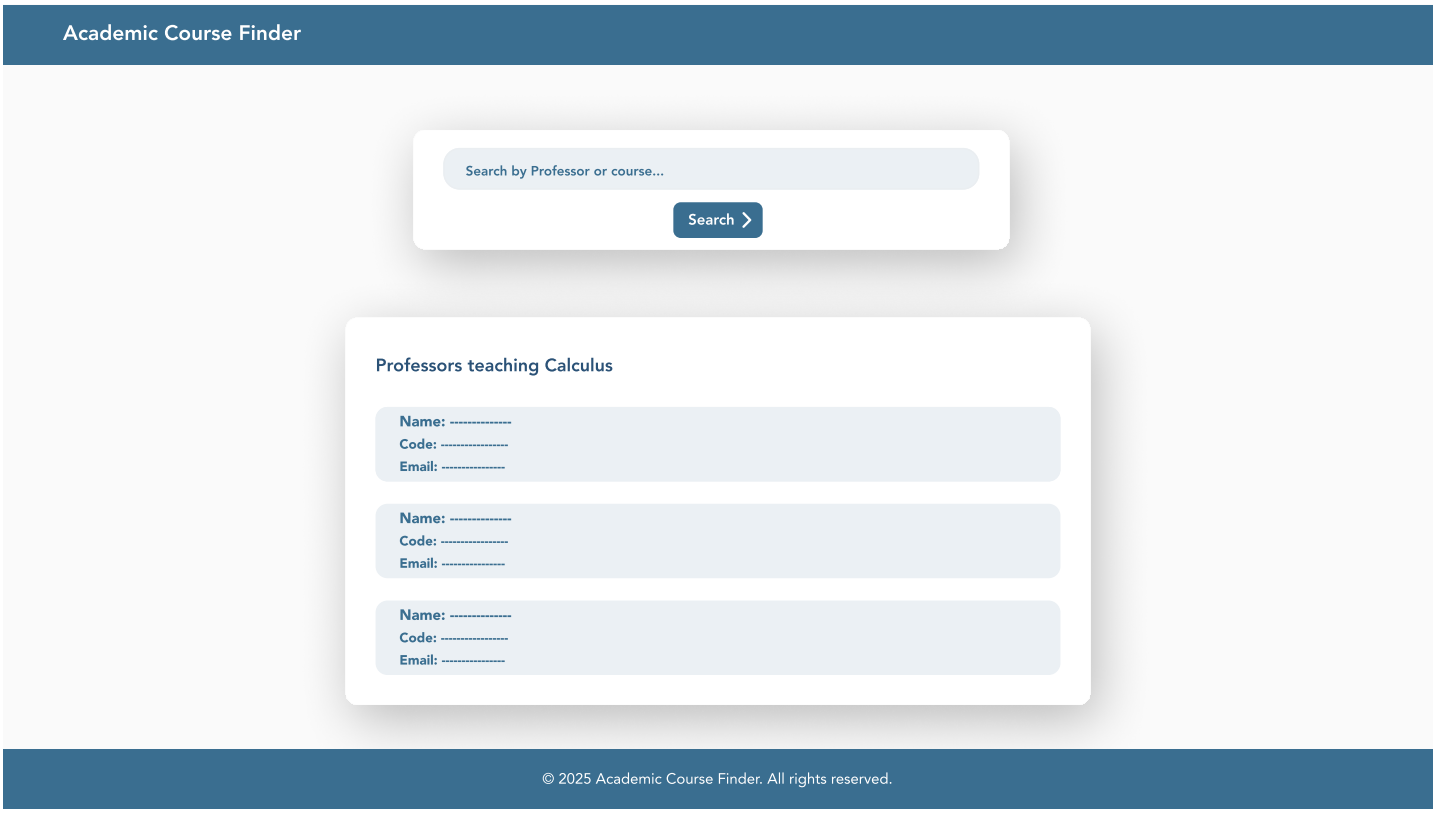


Figure 4: Web User Interface (v1.0)

Figure 5: Web User Interface (v1.0)

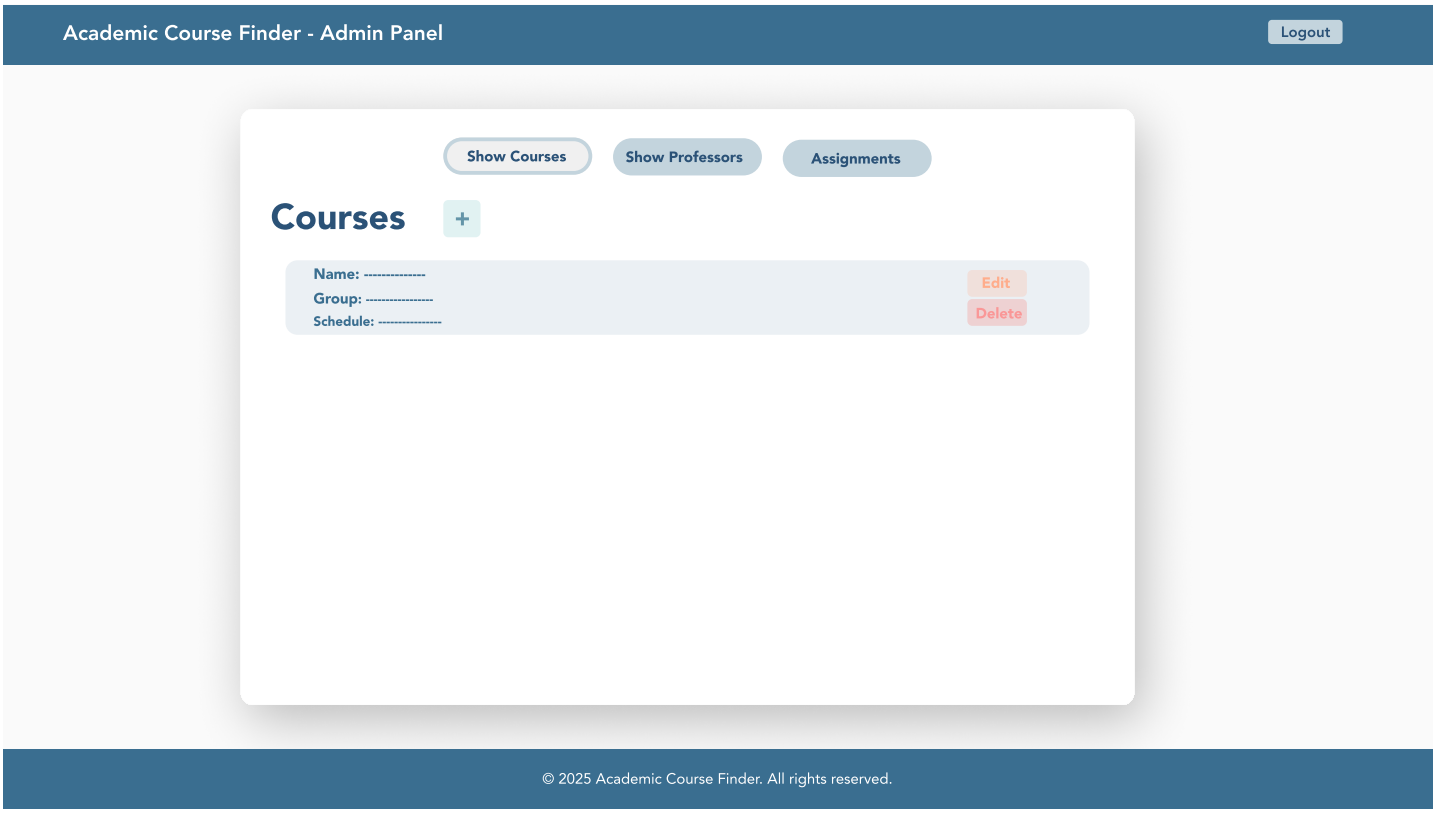
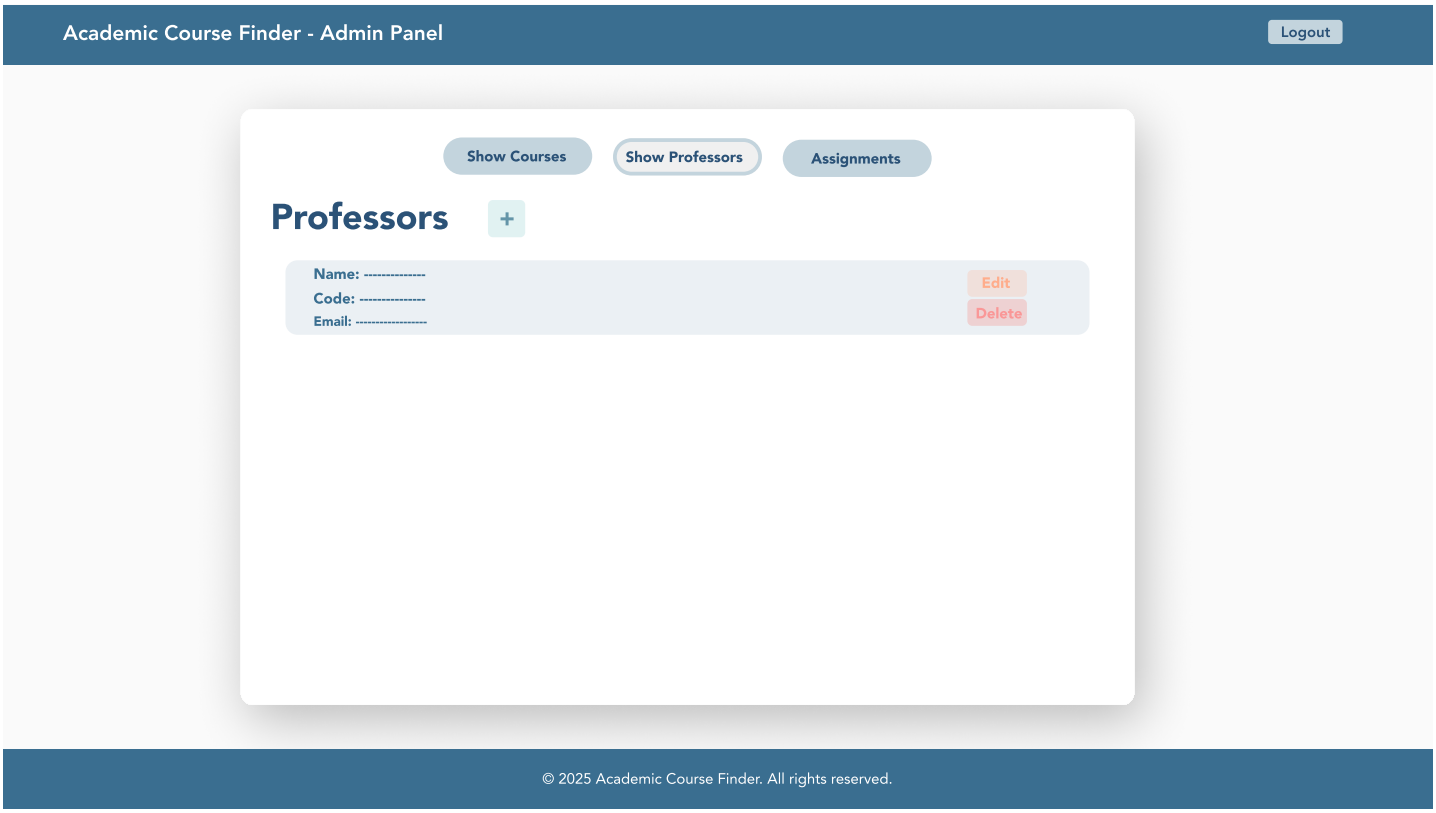


Figure 6: Web User Interface (v1.0)



The user interface was designed following the principle "Simple. Clear. Consistent." The visual design prioritizes clarity, usability, and role-based access control to ensure that each stakeholder group can efficiently accomplish their tasks without unnecessary complexity. Six main views were designed and subsequently implemented in the production environment, representing the complete user journey through the system.

The interface accommodates three distinct user roles with different access levels and capabilities. Anonymous users can access the search functionality, allowing them to query courses by code, title, or professor details without authentication. The search results display comprehensive course information including schedules and associated instructors. Authenticated administrators access additional views for data management, including CRUD operations for courses and professors, as well as the assignment interface that links professors to specific courses and groups.

Design decisions emphasized consistency across views through standardized navigation patterns, uniform typography, and coherent color schemes that distinguish between public and administrative sections. The responsive layout adapts to different screen sizes, ensuring accessibility across desktop and mobile devices. Form validation provides immediate feedback to users, reducing errors during data entry. The role-based design ensures that administrative functions remain hidden from unauthorized users while maintaining an intuitive experience for all stakeholder groups.

The previous figures presents the final implemented views of the web application, demonstrating the translation from initial mockups to functional interfaces. These views validate the usability requirements established during the requirements analysis phase and confirm that the system meets the interaction needs identified in the user stories.

4.3 Architecture Overview

The system architecture implements a microservices approach with clear separation of concerns across three main components: frontend, an authentication service and a business logic service. This architectural decision was mandated by project requirements and offers several strategic advantages for development, deployment, and maintenance.

The separation of authentication into an independent service provides security isolation, ensuring that credential management and token generation occur in a dedicated, hardened environment. This service connects to a database optimized for transaction processing and user session management. The authentication service issues JWT tokens upon successful login, which are subsequently validated by the business logic service for

protected endpoints. This token-based approach enables stateless authentication, simplifying horizontal scaling and load balancing.

The business logic service handles all domain operations including course management, professor administration, search functionality, and relationship assignments. By isolating these operations in a separate service connected to a database different from the one used for authentication, the system achieves better performance through database specialization.

Component communication follows RESTful principles through well-defined API contracts. The frontend makes HTTPS requests to both backend services, with the authentication service providing tokens and the business service consuming them for authorization. This loose coupling allows independent deployment and version management of each service. Furthermore, the architecture supports parallel development, enabling team members to work simultaneously on different services without integration conflicts.

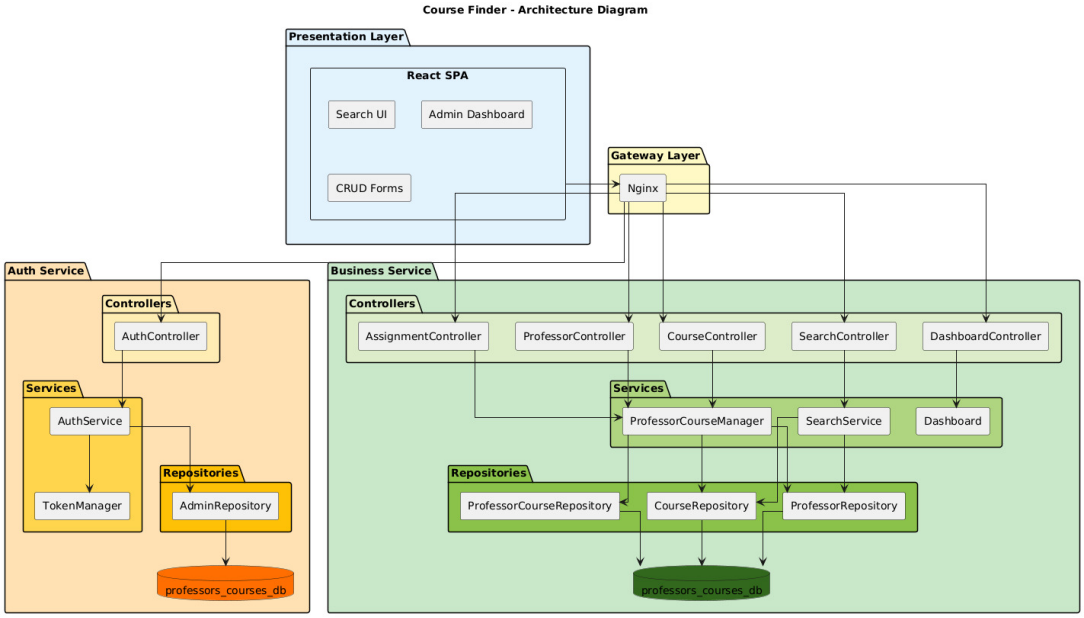


Figure 7: Architecture Diagram (v1.0).

Figure 7 illustrates the high-level architecture, showing the relationships between components, the data flow from user interaction through business logic to persistence layers, and the external interfaces exposed by each service. This diagram serves as the reference model for understanding system boundaries and component responsibilities throughout the development process.

4.4 Data Model and Database Schema

The data model defines four primary entities that capture the essential academic domain: Professor, Course, Admin, and ProfessorCourse. This structure was designed to support the core user requirements identified during the analysis phase, particularly the need to search courses by multiple criteria and maintain clear associations between professors and their teaching assignments.

The Professor entity contains identification attributes (id, username, name) and a specialty field that categorizes the instructor's academic area. This information enables filtering and grouping operations in search queries. The Course entity includes structural attributes (id, code, name) along with academic metadata (credits, category) that support both discovery and academic planning. The unique course code constraint ensures data integrity and prevents duplicate registrations.

The relationship between professors and courses implements a many-to-many association through the ProfessorCourse junction entity. This design decision reflects the reality of academic scheduling where one professor may teach multiple courses and one course may be taught by multiple professors across different groups or semesters. The ProfessorCourse entity includes additional context such as groupId (to distinguish sections), assignedAt (for temporal tracking), and status (to manage active versus historical assignments). This rich association model provides the flexibility required for complex academic scenarios while maintaining referential integrity.

The Admin entity remains separate from the academic domain, containing only authentication-relevant attributes (id, username, passwordHash, email). This separation ensures that administrative access control does not interfere with the core domain logic and allows for independent security policy management.

Design decisions prioritized query performance for the primary use case: searching courses by professor or course attributes and retrieving all related information in minimal database operations. The normalized structure prevents data redundancy while the junction entity enables efficient joins. Repository interfaces abstract database operations, providing a clean separation between domain logic and persistence mechanisms that facilitates future database migration or optimization.

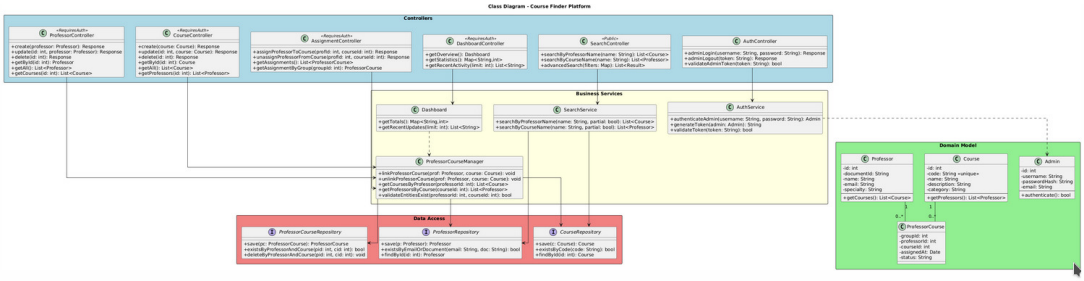


Figure 8: Class diagram (v1.0).

Figure 8 presents the complete class diagram including entities, repositories, services, and controllers across all architectural layers. This diagram demonstrates how the data model integrates with business logic components and exposes functionality through controller endpoints. The layered structure visible in the diagram reinforces the architectural principles of separation of concerns and dependency management established in the system design.

4.5 API Endpoints and Contracts

4.6 Deployment View

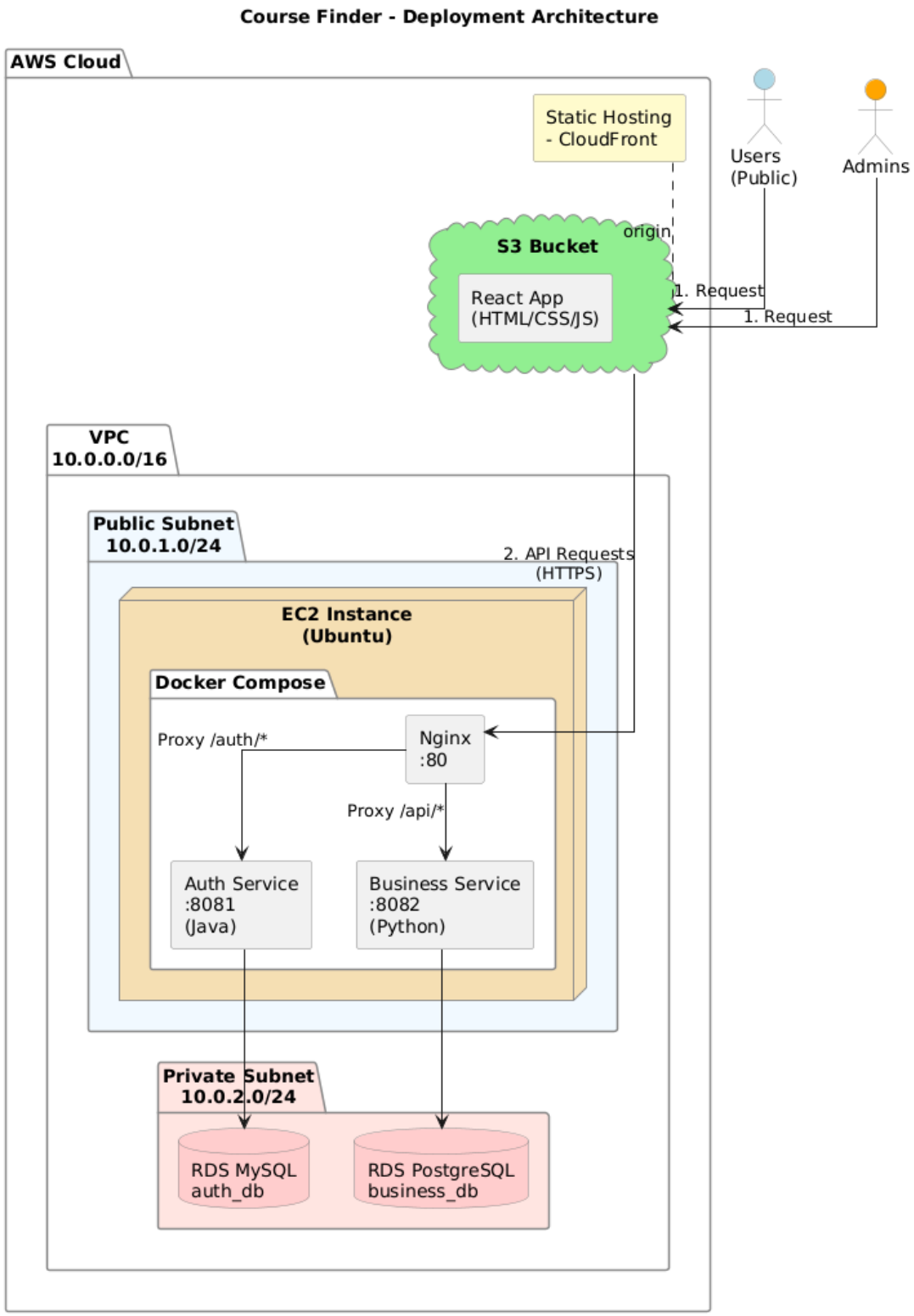


Figure 9: Deployment diagram (v1.0).

5 Implementation and Quality Assurance

5.1 Development Environment and Tools

5.2 Testing Strategy

5.3 CI/CD Pipeline

5.4 Code Quality and Standards

6 Results and Discussion

The outcomes of Workshops 1 and 2 have established a solid analytical foundation for the project. The main deliverables include the problem definition, user stories with acceptance criteria, CRC cards, architectural and deployment diagrams, and early Web UI mockups. These artifacts provide a coherent view of the system's objectives and structure, enabling the team to validate the feasibility of the proposed solution before implementation.

6.1 Testing Results

6.2 System Performance Evaluation

6.3 Achievement of Objectives

6.4 Limitations and Lessons Learned

7 Conclusions

This first report consolidates the analytical and design foundations of the project, demonstrating coherent progress from requirement elicitation to system architecture definition. The established artifacts and planning ensure a clear path toward implementation, testing, and integration in the following workshop stages.

Glossary

API Application Programming Interface.

CI/CD Continuous Integration / Continuous Deployment.

JWT JSON Web Token.

CRC Class - Responsability - Collaboration Cards